Predicting Market Decisions via Continuous Return Forecasting with Risk-Aware Decision Mapping

Student number: 22033917
GitHub repository: https://github.com/UCL-ELEC0136/10-final-assessment-xzzc666

1 Introduction

2 System overview

3 Data Discovery and Acquisition

3.1 Data Sources and Usage Protocols

This project uses **FRED** (Federal Reserve Economic Data) as the primary data source. FRED provides authoritative macroeconomic and financial time-series data and supports programmatic access via a free API key.

According to the official *FRED API Terms of Use* **?**, the free API may be used for academic and non-commercial research purposes. All retrieved data in this study are used exclusively for model training and analysis and are neither redistributed nor shared, fully complying with the stated usage conditions.

Data acquisition is conducted through the official FRED API documentation **?**, with primary reference to the *series observations* endpoint **?**. HTTP requests are issued using Python's `requests` library, with parameters including `api_key`, `series_id`, `file_type`, and `realtime_start`, and `realtime_end`. This design enables automated and reproducible data collection.

3.2 Dataset Selection

A total of **31 distinct time-series datasets** are incorporated in this study. These indicators are grouped into economically meaningful categories, including equity markets, volatility and systemic risk, credit spreads, interest rates and term structure, inflation, liquidity conditions, labour market, economic activity, housing market, and consumption and sentiment. The complete list and selection rationale are summarised in Table 3 and reported in detail in the Appendix.

The dataset spans the period from January 1, 2015 to December 31, 2025, covering multiple market regimes and macroeconomic cycles.

Using a limited number of input variables may lead to overfitting under high-capacity model configurations. To mitigate this risk, a broad set of macro-financial indicators is collected to provide richer contextual information. Not all indicators are directly fed into the model simultaneously; instead, the full dataset serves as a candidate feature pool from which informative representations are selectively extracted during subsequent data processing and representation learning stages.

3.3 Data Backup and Logging

To improve acquisition efficiency and robustness, a local caching mechanism is introduced to avoid repeated downloads of identical historical data. All raw data are persistently stored in a local SQLite database (`data_raw.db`) under the `data` directory (details showed in Section 4.3). When data already exist locally, they are retrieved directly without issuing new API requests, thereby reducing acquisition time and preventing access rate limitations.

The overall acquisition workflow is illustrated in Figure 1. A control hyperparameter `REGENERATE` determines whether raw data should be re-fetched. When enabled, or when the local database is absent, the system automatically identifies missing series and requests only the unavailable components from the FRED API.

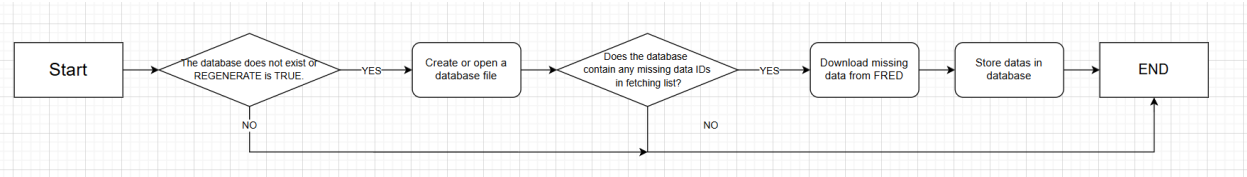


Figure 1: Data acquisition and storage workflow

To support runtime monitoring, a lightweight logging mechanism records data acquisition progress and execution errors in a dedicated log file (`data_fetch.log`), thereby improving the observability and reliability of the data collection pipeline. Subsequent processing and representation stages are likewise tracked through corresponding log files, including `data_processing.log` and `data_representing.log`, ensuring end-to-end traceability across the entire data pipeline.

4 Object Model, Data Validation, and Storage

4.1 Data Structure and Organisation

Each data series retrieved from the API follows a unified tabular structure consisting of five fields: `series_id`, `date`, `value`, `realtime_start`, and `realtime_end`. While this format preserves complete release information, it is not directly suitable for downstream model training and therefore requires structural reorganisation.

The fields `realtime_start` and `realtime_end`, which describe data release windows, do not contribute to feature construction and are removed during preprocessing. All variables are originally stored in `text` format; consequently, `value` is converted to numerical format and `date` to a date-type index to enable reliable temporal alignment and numerical computation.

Furthermore, storing `series_id` repeatedly across rows is unnecessary. Since its primary role is to distinguish indicators, it is restructured as the column dimension of the dataset. This transformation reorganises the dataset by using `series_id` solely as column names, allowing different indicators to be clearly distinguished without redundant repetition across rows.

As all time series span a unified period from January 1, 2015 to December 31, 2025, different indicators are aligned by the date index, resulting in a single multivariate dataset. The final schema is shown in Table 1.

Table 1: Dataset schema

Date	SP500	DJIA	NASDAQCOM	VIXCLS	TEDRATE	STLFSI4	$\cdots$
datetime64[us]	float64	float64	float64	float64	float64	float64	float64

4.2 Data Validation

During dataset construction, observations sharing identical timestamps are automatically aligned across all indicators. Any inconsistencies that prevent proper alignment trigger runtime errors and are recorded in log files, ensuring temporal integrity.

Missing values arise primarily from heterogeneous data release frequencies (daily, weekly, and monthly) and occasional publication suspensions during holidays or exceptional events. To address this issue, a forward-fill strategy is applied uniformly, whereby missing entries are filled using the most recent available observation. This approach reflects realistic information availability and avoids the introduction of future data, thereby preventing data leakage. Linear interpolation is deliberately avoided, as it implicitly relies on future observations and may distort temporal dynamics.

After forward filling, a small number of missing values may remain at the beginning of the time series due to the absence of prior observations. As this segment accounts for fewer than 30 days and lies entirely at the start of the timeline, it is removed without affecting data continuity.

In addition, no explicit outlier removal or value clipping is performed. Extreme observations are retained, as such values often reflect periods of market stress, regime shifts, or systemic events. Removing or smoothing these values may eliminate informative signals that are critical for modelling financial dynamics.

4.3 Data Storage

An SQLite database is adopted as the storage backend due to its lightweight design, native Python support, and suitability for offline operation. Since all indicators are collected once and reused for long-term experimentation, the system does not require concurrent access or frequent write operations, making a local relational database sufficient.

A structured SQL schema is preferred over NoSQL alternatives to enforce strict temporal alignment and fixed field definitions, thereby improving dataset stability and consistency.

To support modular processing and reproducibility, datasets are persistently stored at each major stage of the pipeline: raw API outputs in `data_raw.db`, cleaned and validated data in `data_processed.db`, and model-ready feature representations in `data_represented.db`. This staged storage design enables independent inspection, rollback, and reuse of intermediate results.

5 Dataset Construction and Sampling

5.1 Feature Construction

Based on the cleaned dataset, additional derived variables are constructed to support market modelling.

For the equity indices `SP500`, `DJIA`, and `NASDAQCOM`, logarithmic returns are computed as

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right), \tag{1}$$

$$\mathbf{x}_t = \left[\mathbf{x}_t^{\text{raw}}\,;\,\mathbf{r}_t\,;\,\overline{P}^{(30)}_{t,\text{SP500}}\,;\,\overline{r}^{(30)}_{t,\text{SP500}}\right], \tag{2}$$

which provide a scale-invariant representation of relative price movements.

As the S&P 500 index serves as the primary prediction target, rolling statistics over the previous 30 trading days are additionally computed for both its price level and logarithmic return, including rolling mean price and rolling mean return. These smoothed features enhance stability.

All derived variables are computed exclusively from historical observations, thereby preventing data leakage.

5.2 Temporal Sampling and Dataset Splitting

$$\mathbf{X}_t = \left[\mathbf{x}_{t-49}, \mathbf{x}_{t-48}, \ldots, \mathbf{x}_t\right] \tag{3}$$

Learning samples are constructed using a sliding-window approach, where the past 50 trading days are used as input to predict market behaviour over the subsequent 20 trading days. A longer prediction horizon is adopted due to the high stochasticity and limited predictability of short-term financial price fluctuations.

The dataset is partitioned chronologically into training, validation, and test sets with a ratio of 70%, 15%, and 15%, respectively. Gaussian normalisation is then applied to all numerical variables, with mean and variance estimated exclusively from the training set and reused for the validation and test sets, ensuring consistent scaling without information leakage.

5.3 Target Definition

The model is designed to forecast future market behaviour by predicting a continuous return value rather than directly outputting discrete trading actions. Specifically, the forecasting target is defined as the cumulative logarithmic return over the next 20 trading days:

$$Y_t^R = \sum_{i=1}^{20} r_{t+i}. \tag{4}$$

For evaluation and decision interpretation, the predicted cumulative return is subsequently mapped to discrete trading actions through a threshold-based decision function. The same transformation is applied to the ground-truth target values, ensuring consistent comparison at the decision level:

$$Y_t^C = \begin{cases} \text{Buy}, & Y_t^R > \theta_2, \\ \text{Hold}, & \theta_1 < Y_t^R \leq \theta_2, \\ \text{Sell}, & Y_t^R \leq \theta_1, \end{cases} \tag{5}$$

Compared with direct discrete classification, this formulation provides richer training feedback during optimisation, as the learning process is guided by the magnitude of prediction errors rather than solely by categorical correctness. This enables more informative gradient updates and improves optimisation stability. At the same time, prediction errors are more likely to manifest as neighbouring decision deviations (e.g., Buy vs. Hold or Hold vs. Sell), while severe directional reversals (Buy vs. Sell) are less frequent.

The limitations of direct discrete classification are further analysed in Appendix 10.1, where comparative experiments demonstrate severe overfitting and unstable optimisation behaviour.

Thresholds set to $\theta_1 = 0$ and $\theta_2 = 0.03$. This asymmetric threshold design prioritises risk avoidance over return maximisation, reflecting the practical objective of reducing false positive buy signals. In financial decision-making, avoiding losses is often more critical than capturing marginal gains, making conservative classification boundaries preferable.

The resulting label distributions across training, validation, and test sets are summarised in Table 2. Overall, the class proportions remain relatively balanced, reducing the risk of biased training and enabling fair evaluation of classification performance.

Table 2: Label distribution across dataset splits

	Buy	Hold	Sell
Train	502 (28.3%)	717 (40.4%)	557 (31.4%)
Val	150 (39.5%)	123 (32.4%)	107 (28.2%)
Test	120 (31.4%)	161 (42.1%)	101 (26.4%)

6 Problem formalisation

6.1 Loss Function

The forecasting model is trained to predict continuous future cumulative returns. Accordingly, a loss function is required to quantify and penalise the discrepancy between predicted and realised returns.

In this study, the Smooth L1 loss is adopted as the optimisation objective:

$$\mathcal{L} = \begin{cases} \frac{1}{2}(\hat{Y}_t - Y_t^R)^2, & |\hat{Y}_t - Y_t^R| < 1, \\ |\hat{Y}_t - Y_t^R| - \frac{1}{2}, & \text{otherwise}, \end{cases} \tag{6}$$

where $\hat{Y}_t$ and Y_t^R denote the predicted and ground-truth cumulative logarithmic returns, respectively.

The Smooth L1 loss combines the advantages of mean squared error and mean absolute error. For small prediction deviations, it behaves quadratically, providing smooth gradients that facilitate stable convergence. For larger deviations, it transitions to a linear form, reducing sensitivity to extreme prediction errors.

This property is particularly desirable in financial time-series forecasting, where return distributions are heavy-tailed and occasional extreme market movements are common. By limiting the influence of large outliers on gradient updates, Smooth L1 loss prevents unstable optimisation while still preserving meaningful error magnitude information.

6.2 Evaluation Metrics

To evaluate model performance, three complementary metrics are adopted:

- **Accuracy.** Measures the proportion of correctly predicted trading actions:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{y}_i = y_i), \tag{7}$$

  where N denotes the total number of samples, y_i and $\hat{y}_i$ represent the ground-truth and predicted class labels, respectively, and $\mathbb{I}(\cdot)$ is the indicator function.

- **Micro-averaged F1 score.** Computes precision and recall globally by aggregating true positives (TP), false positives (FP), and false negatives (FN) across all classes:

$$\text{F1}_{\text{micro}} = \frac{2\sum_c \text{TP}_c}{2\sum_c \text{TP}_c + \sum_c \text{FP}_c + \sum_c \text{FN}_c}. \tag{8}$$

  This formulation provides a stable evaluation under moderate class imbalance.

- **Directional reversal rate.** Measures the frequency of severe decision errors where `Buy` and `Sell` predictions are reversed:

$$\text{DRR} = \frac{\sum_{i=1}^{N}\mathbb{I}\big((y_i = \text{Buy} \wedge \hat{y}_i = \text{Sell}) \vee (y_i = \text{Sell} \wedge \hat{y}_i = \text{Buy})\big)}{N}. \tag{9}$$

In addition to scalar metrics, confusion matrices are used to visualise classification behaviour. This representation jointly reflects overall accuracy, class-wise error patterns, and directional reversals, enabling intuitive interpretation of model performance.

7 Representation Learning, Feature Design and Impact Analysis

7.1 Representation Learning and Feature Influence

The constructed dataset contains a large number of macroeconomic indicators, market variables, and derived features, resulting in a high-dimensional input space. Such dimensionality increases computational cost and may negatively affect model generalisation due to the curse of dimensionality. Representation learning is therefore introduced to obtain a more compact and efficient feature representation.

7.2 Choice of Dimensionality Reduction Method

Principal Component Analysis (PCA) is adopted to project the original feature space into a lower-dimensional latent representation by preserving directions of maximum variance. This enables effective removal of redundant information while maintaining the dominant structure of the data.

Alternative feature reduction approaches are not adopted for the following reasons:

- **Correlation-based analysis.** The large number of variables and complex interdependencies among macro-financial indicators make pairwise correlations difficult to interpret reliably.
- **Wrapper-based feature selection.** Such methods require repeated model training and become computationally prohibitive under high-dimensional input spaces.

The objective of representation learning in this study is not to interpret individual feature contributions, but to eliminate redundant dimensions and improve modelling efficiency. PCA is therefore well suited to this setting.

7.3 PCA Compression Strategies

Two PCA-based compression strategies are considered. The first projects the original feature space onto the subspace spanned by the eigenvectors corresponding to the top 10 largest eigenvalues. The second retains principal components whose eigenvalues exceed a predefined threshold of 0.1, allowing the retained dimensionality to be determined adaptively by the intrinsic variance structure of the data.

In this study, the second strategy is adopted as the primary compression method, as it preserves informative variance while discarding weakly informative components in a data-driven manner.

7.4 Impact Analysis

After projection, the input representation is reduced to 13 principal components. The relationships between the principal components and the predicted target Y_t^R are provided in Appendix 7.

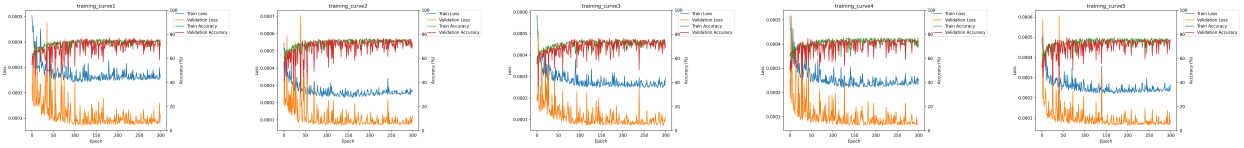
Empirical inspection indicates that several components, including PCA3, PCA6, PCA8, and PCA10, exhibit positive correlation with the target variable, while PCA5 shows a more pronounced negative correlation. These patterns suggest that the transformed feature space captures meaningful directional information and improves linear separability with respect to the prediction target.

Overall, PCA-based dimensionality reduction substantially reduces feature dimensionality, mitigates the risk of dimensionality explosion, and leads to improved training efficiency and optimisation stability in downstream learning models.

8 Training and evaluation protocol

The effect of different dataset and parameter configurations on training stability was systematically analysed. Detailed comparisons of training dynamics under varying feature sets are provided in Appendix 10.2.

8.1 Training Curves and Model Selection



(a) Seed 0 (b) Seed 1 (c) Seed 2 (d) Seed 3 (e) Seed 4

Figure 2: Training and validation curves under five different random seeds.

Figure 2 presents the training dynamics obtained under five different different seeds (0–4). For clarity of visual inspection, enlarged training curves under different random seeds are provided in the Appendix Figure 8. Across all runs, the model demonstrates highly consistent convergence behaviour, indicating strong robustness to random initialisation and good generalisation capability.

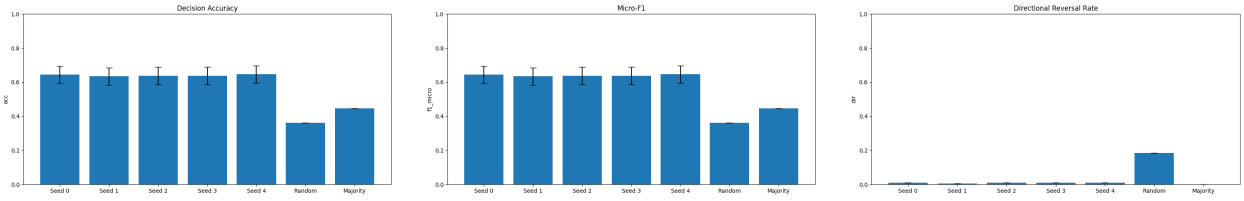
Although noticeable oscillations appear during early training, the fluctuation magnitude gradually decreases after approximately 100 epochs, and the optimisation process becomes increasingly stable. This suggests that the model successfully learns meaningful temporal structures while progressively reducing sensitivity to noise.

To ensure reliable model selection, checkpoint saving is enabled after 200 epochs. Among these later epochs, the model achieving the highest validation accuracy is selected as the final model and subsequently evaluated on the test set. This strategy mitigates the effect of transient fluctuations and ensures that testing is conducted using the most stable and performant parameter configuration.

8.2 Evaluation Results and Baseline Comparison

Due to space limitations, detailed visualisations are deferred to Appendix Section 11.2.

The proposed model is evaluated on the test set under five different random seeds. Performance is measured using decision accuracy, micro-averaged F1 score, and directional reversal rate (DRR). Two baseline strategies are included for comparison: a random decision policy and a majority-class policy.



(a) Accuracy (b) Micro-F1 (c) Directional reversal rate

Figure 3: Evaluation metrics across five random seeds and baseline methods.

As shown in Figure 3a, the proposed model achieves stable performance across different random seeds, with accuracy consistently around 65% and only minor variation. In contrast, the random baseline achieves approximately 33% accuracy, while the majority baseline reaches about 42%, consistent with the class distribution of the test set. Even at the lower bound of the confidence interval, the proposed model consistently outperforms both baselines, indicating statistically robust improvements.

A similar trend is observed for the micro-averaged F1 score, as shown in Figure 3b, further confirming the consistency of the model's decision-level performance.

(a) Proposed model (5 seeds) (b) Majority baseline (c) Random baseline

Figure 4: Overview of confusion matrices on the test set.

The confusion matrices in Figure 4 demonstrate well-structured classification behaviour. Most prediction errors occur between neighbouring actions (e.g., `Buy` vs. `Hold`), while direct `Buy`–`Sell` reversals are extremely rare.

As shown in Figure 3c, the proposed model maintains an extremely low DRR across all random seeds, substantially lower than the random baseline. Although the majority baseline achieves zero DRR, this result is degenerate, as it arises from always predicting the `Hold` class rather than from informed decision-making.

Overall, the proposed model achieves significantly higher predictive performance than baseline strategies, while simultaneously maintaining strong decision-level safety.

9 References

10 Appendix A Additional Analysis

10.1 Analysis of Direct Discrete Classification

To further justify the target formulation adopted in this study, an additional set of experiments was conducted using a direct discrete classification model. This appendix provides a comparative analysis illustrating the limitations of directly predicting trading actions, and motivates the use of continuous-return forecasting followed by threshold-based decision mapping.

10.1.1 Model Formulation

In the direct classification setting, the model outputs discrete trading actions (`Buy`, `Hold`, `Sell`) directly. The optimisation objective is defined using the standard cross-entropy loss:

$$\mathcal{L}_{\text{cls}} = -\log \frac{e^{z_y}}{\sum_j e^{z_j}}, \tag{10}$$

where z_j denotes the predicted logit for class j, and y is the ground-truth trading label.

Unlike the continuous-return formulation used in the main experiments, this approach provides only categorical feedback during training, indicating whether a prediction is correct or incorrect, without conveying the magnitude of prediction error.

10.1.2 Experimental Setup

For fair comparison, the direct classification model was evaluated under identical experimental conditions to those described in the main body of this report. Specifically, the same preprocessing pipeline, derived feature construction, and PCA-based dimensionality reduction (retaining components with eigenvalues greater than 0.1) were applied.

Three dataset configurations were tested:

- **Full dataset:** 31 macro-financial indicator categories and their derived features.
- **Reduced dataset:** 9 selected indicator categories and corresponding derivatives.
- **Minimal dataset:** the S&P 500 index and its derived features only.

All models were trained using the same network architecture and optimisation settings as the continuous forecasting model.

10.1.3 Training Behaviour

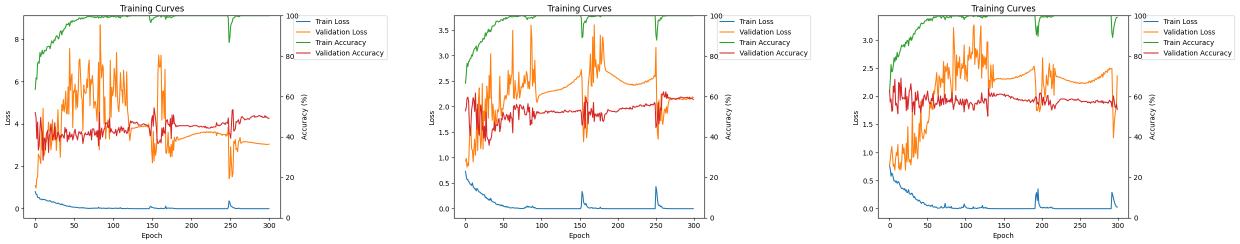
The resulting training curves are shown in Figure 5.

Across all dataset configurations, the training accuracy rapidly approaches nearly 100%, while the validation accuracy remains significantly lower. This behaviour indicates severe overfitting, even when the dataset size and feature diversity are increased.

Notably, enlarging the dataset does not effectively alleviate this issue. Although the classification model consistently outperforms random and majority baselines, its decision accuracy, convergence speed, and training stability remain substantially inferior to those achieved by the continuous-return forecasting formulation.

10.1.4 Discussion

The observed behaviour highlights a fundamental limitation of direct discrete classification in financial decision modelling.



(a) Full dataset (b) Reduced dataset (c) SP500-only dataset

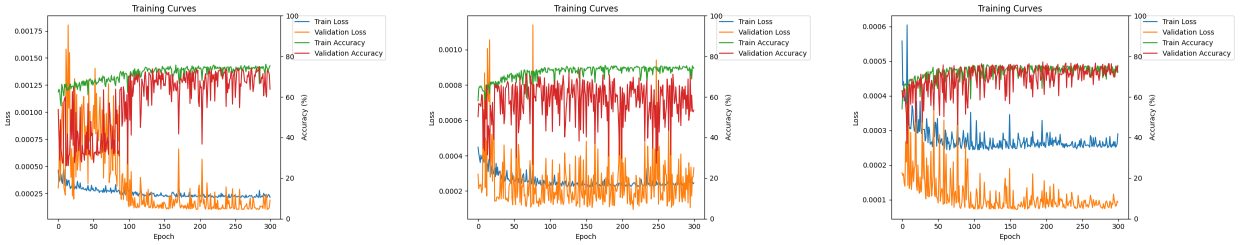
Figure 5: Training and validation curves for direct discrete classification.

Because the cross-entropy loss only penalises categorical errors, the optimisation process lacks information about how far an incorrect prediction deviates from the true market outcome. As a result, gradient updates are coarse and insensitive to error magnitude, leading to unstable optimisation and poor generalisation.

In contrast, predicting continuous cumulative returns enables the model to receive fine-grained feedback proportional to prediction deviation. This facilitates smoother gradient updates, improved convergence stability, and reduces the likelihood of severe directional errors such as `Buy`–`Sell` reversals.

These results empirically support the modelling choice adopted in this study: continuous return forecasting combined with threshold-based decision mapping provides a more stable and informative learning objective than direct discrete classification.

10.2 Training Stability under Different Dataset Configurations



(a) Full 31-category dataset (b) Reduced 9-category dataset (c) SP500-only dataset

Figure 6: Training curves under different dataset configurations.

As shown in Figure 6a, although both training and validation accuracy converge to satisfactory levels, and the validation loss even falls below the training loss in later epochs, the learning curves exhibit pronounced high-frequency fluctuations. Similar oscillatory behaviour is observed in both accuracy and loss trajectories.

To mitigate this instability, several regularisation strategies were evaluated, including weight decay and gradient clipping. However, these techniques did not substantially reduce the observed oscillations.

This suggests that the model is able to learn meaningful patterns, while the instability is more likely caused by noise and redundancy in the input data rather than optimisation failure.

To investigate this hypothesis, three dataset configurations were evaluated under identical preprocessing settings described in the previous sections, including derived feature construction and PCA-based dimensionality reduction with eigenvalues greater than 0.1:

- **Full dataset:** 31 macro-financial indicator categories and their derived features.3
- **Reduced dataset:** 9 selected indicator categories and corresponding derivatives.4

- **Minimal dataset:** the S&P 500 index and its derived features only.

As illustrated in Figure 6, interestingly, the SP500-based dataset consistently exhibits the fastest convergence, reduced training volatility, and higher decision-level accuracy. This indicates that incorporating a large number of auxiliary indicators may introduce additional noise and representation complexity, thereby weakening effective signal extraction.

Based on these empirical findings, the SP500-based dataset is adopted for all subsequent experiments, as it provides the most stable training behaviour and best overall performance.

11 Appendix B Data Display

Table 3: 31 Categories Dataset

CATEGORY	INDICATORS	REASON
Target market	SP500	Serves as the prediction target, providing the reference equity dynamics for supervised learning
Auxiliary equity markets	DJIA, NASDAQCOM	Capture cross-market co-movement and sectoral divergence that may provide contextual signals for SP500 price dynamics
Volatility and systemic risk	VIXCLS, TEDRATE, STLFSI4	Provide forward-looking information on market uncertainty, which may influence risk premia and short-term SP500 fluctuations
Credit spreads	BAMLH0A0HYM2, BAMLC0A0CM, BAMLC0A1CAAA, BAMLC0A4CBBB	Reflect changes in financing conditions and investor risk appetite that can affect equity valuation and market drawdowns
Interest rates and term structure	GS3M, GS2, GS10, GS30, T10Y2Y, T10Y3M	Encode monetary policy stance and yield curve expectations that influence discount rates applied to SP500 constituents
Inflation	CPIAUCSL, CPILFESL, PCEPI	Capture inflation dynamics that shape policy expectations and indirectly impact equity valuation levels
Liquidity and monetary conditions	M2SL, WALCL	Represent liquidity expansion or contraction that may amplify or suppress SP500 market momentum
Labour market	UNRATE, PAYEMS	Indicate employment conditions affecting household income, consumption capacity, and earnings expectations of SP500 firms
Economic activity	INDPRO, TCU, DGORDER	Describe real economic momentum that may influence corporate profitability and medium-term equity trends
Housing market	CSUSHPISA, HOUST, PERMIT	Reflect interest-rate-sensitive economic sectors that often respond early to policy shifts impacting broader equity markets
Consumption and sentiment	UMCSENT, RSAFS	Provide behavioural signals of household expectations that may affect equity demand and short-term market sentiment

Table 4: 9 Categories Dataset

CATEGORY	INDICATORS	REASON
Target market	SP500	Serves as the prediction target, providing the core equity price dynamics for supervised learning
Auxiliary equity markets	DJIA, NASDAQCOM	Provide complementary market context and capture cross-market co-movement that may influence SP500 short-term dynamics
Volatility and systemic risk	VIXCLS, TEDRATE	Reflect market uncertainty and funding stress conditions that often precede changes in equity risk sentiment
Credit spreads	BAMLH0A0HYM2, BAMLC0A0CM	Represent variations in credit risk perception and financing conditions affecting equity market stability
Interest rates	GS3M, GS2	Capture short- and medium-term interest rate movements that influence discounting behaviour and equity valuation

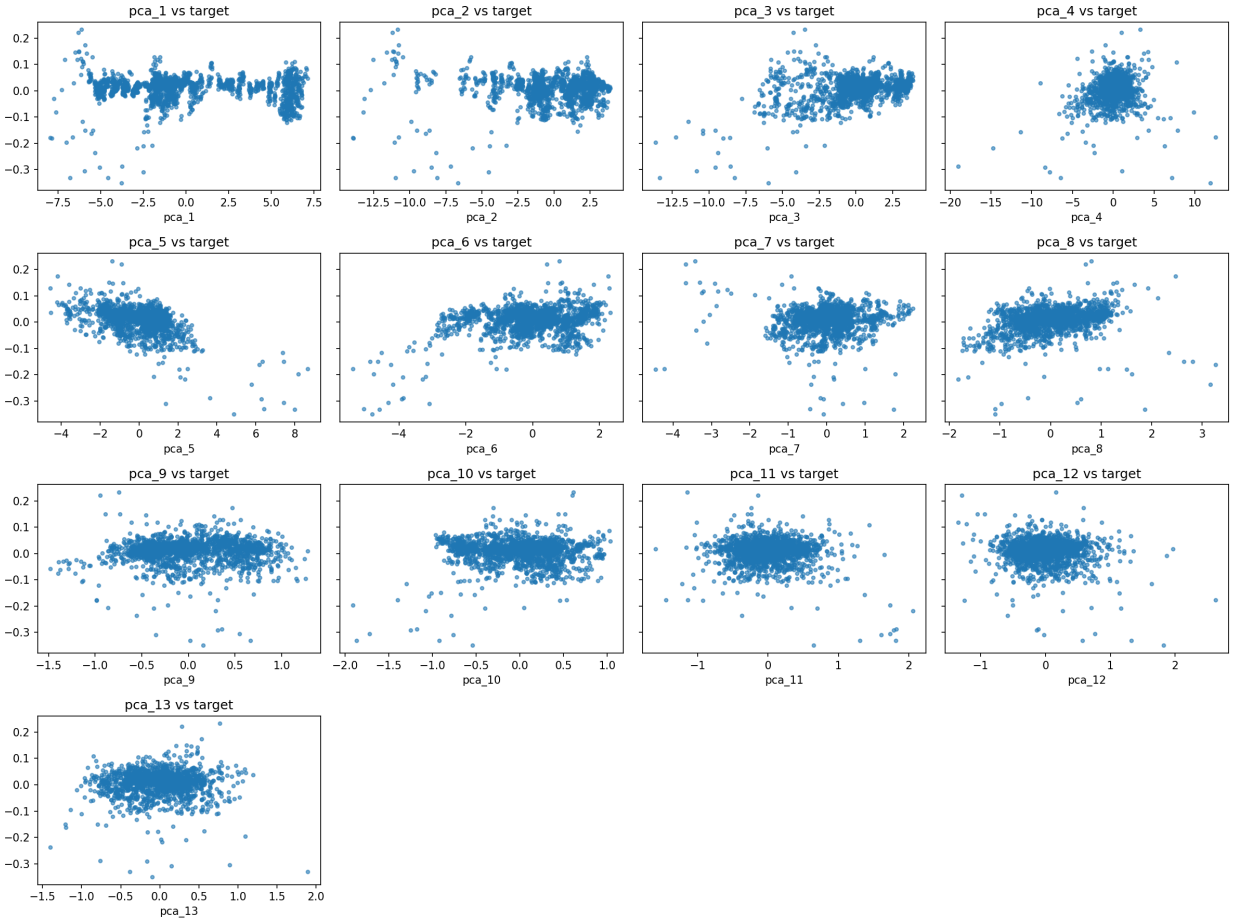
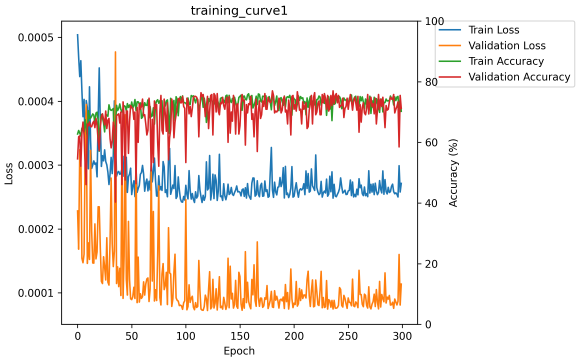


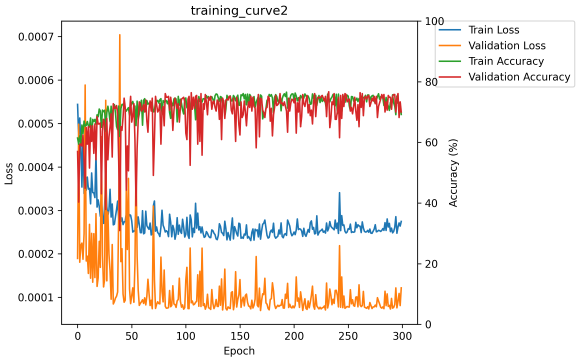
Figure 7: Scatter plots illustrating the relationship between individual PCA components and the predicted target Y_t^R. Each subplot shows one principal component against the future cumulative return.

11.1 Training Curves under Different Random Seeds

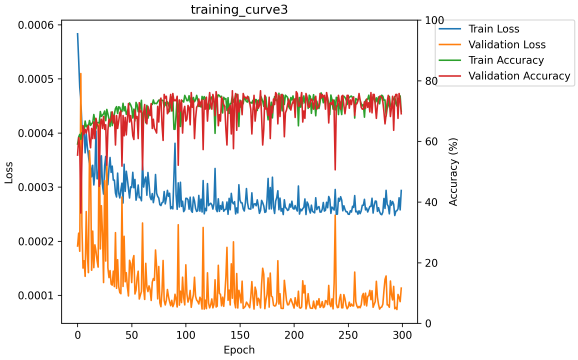
Figure 8 presents enlarged training and validation curves obtained under five different random seeds (0–4). The consistent convergence patterns across different initialisations further confirm the robustness and stability of the proposed model.



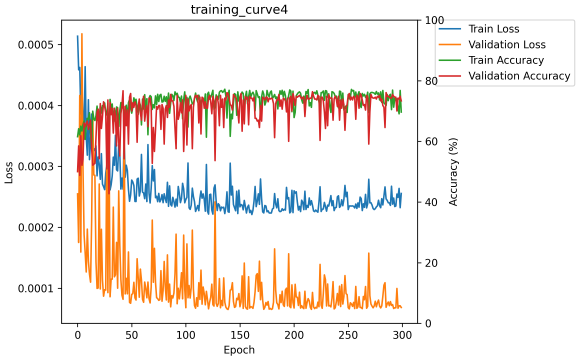
(a) Seed 0



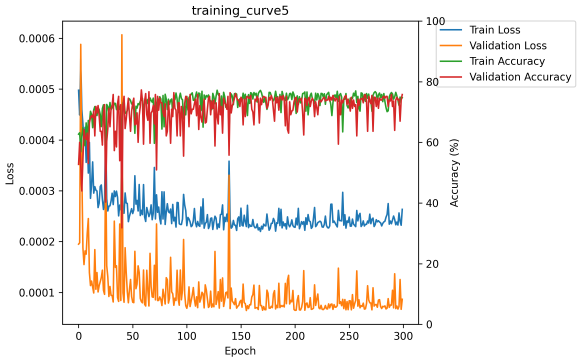
(b) Seed 1



(c) Seed 2



(d) Seed 3



(e) Seed 4

Figure 8: Enlarged training and validation curves under five different random seeds.

11.2 Evaluation Metrics

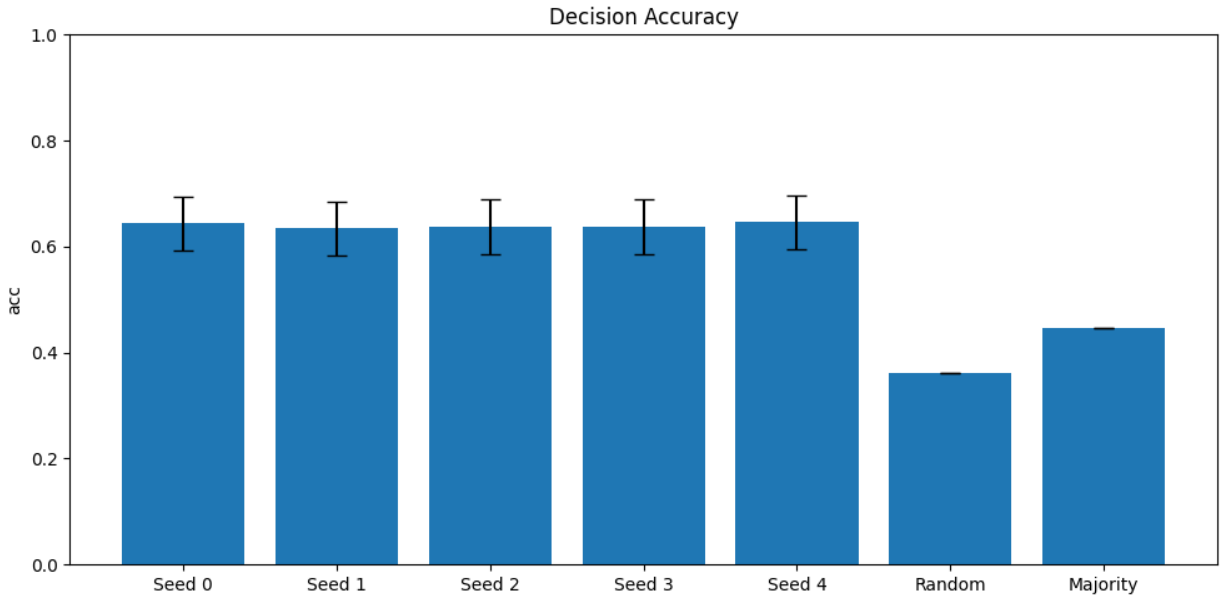


Figure 9: Decision accuracy across five random seeds and baseline methods.

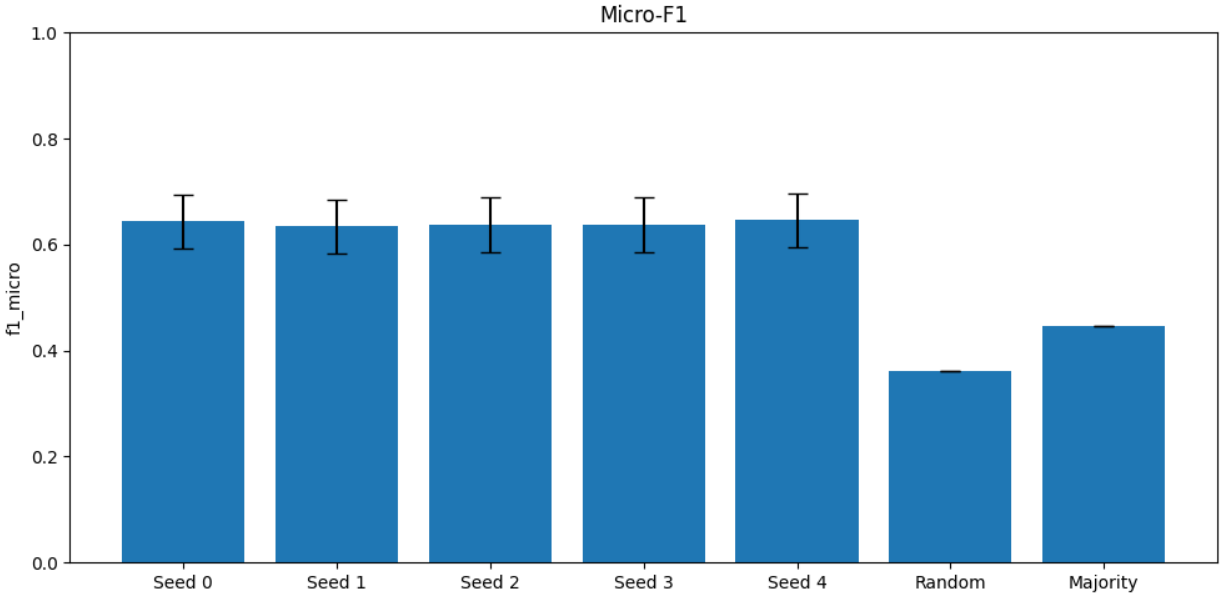


Figure 10: Micro-averaged F1 score across different random seeds and baseline methods.

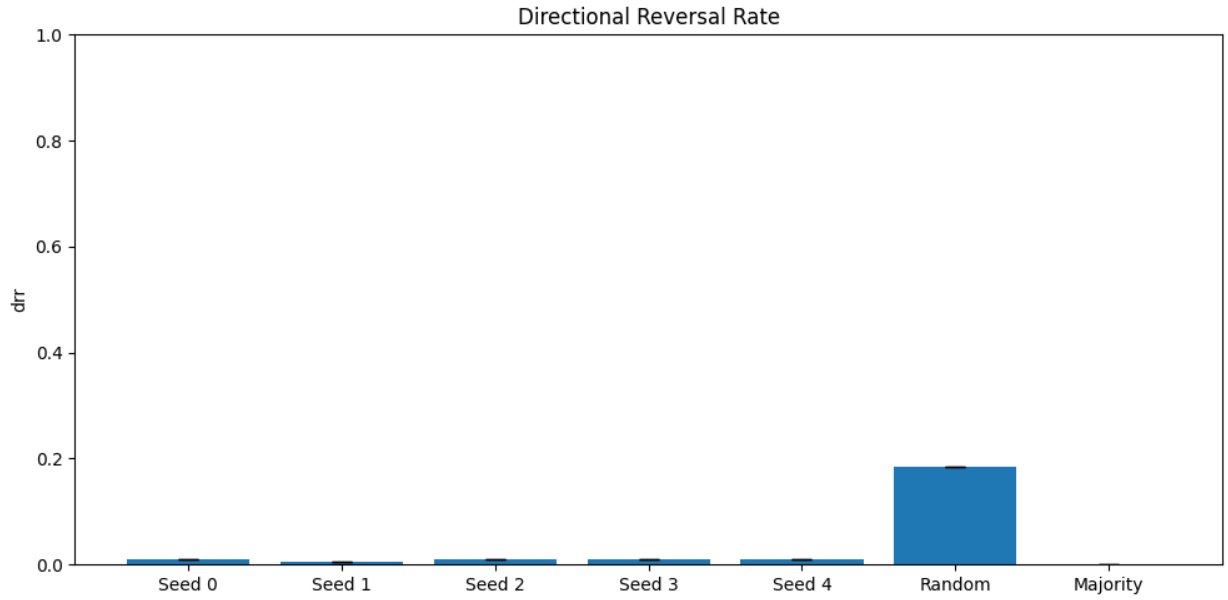


Figure 11: Directional reversal rate (DRR) across random seeds and baseline methods.

11.3 Confusion Matrices

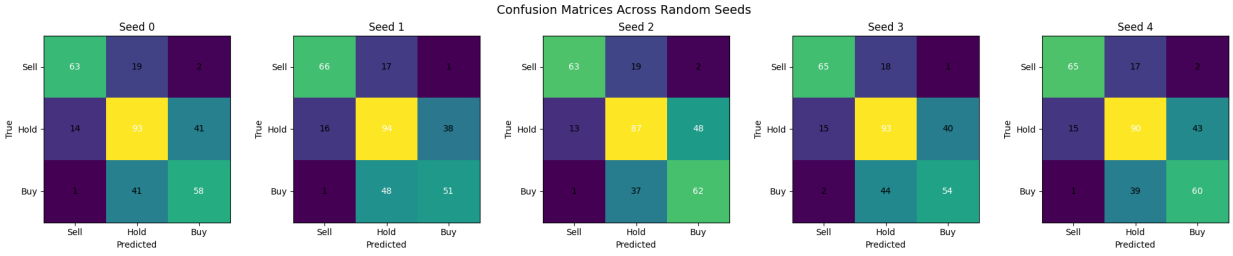


Figure 12: Confusion matrices of the proposed model under five different random seeds.

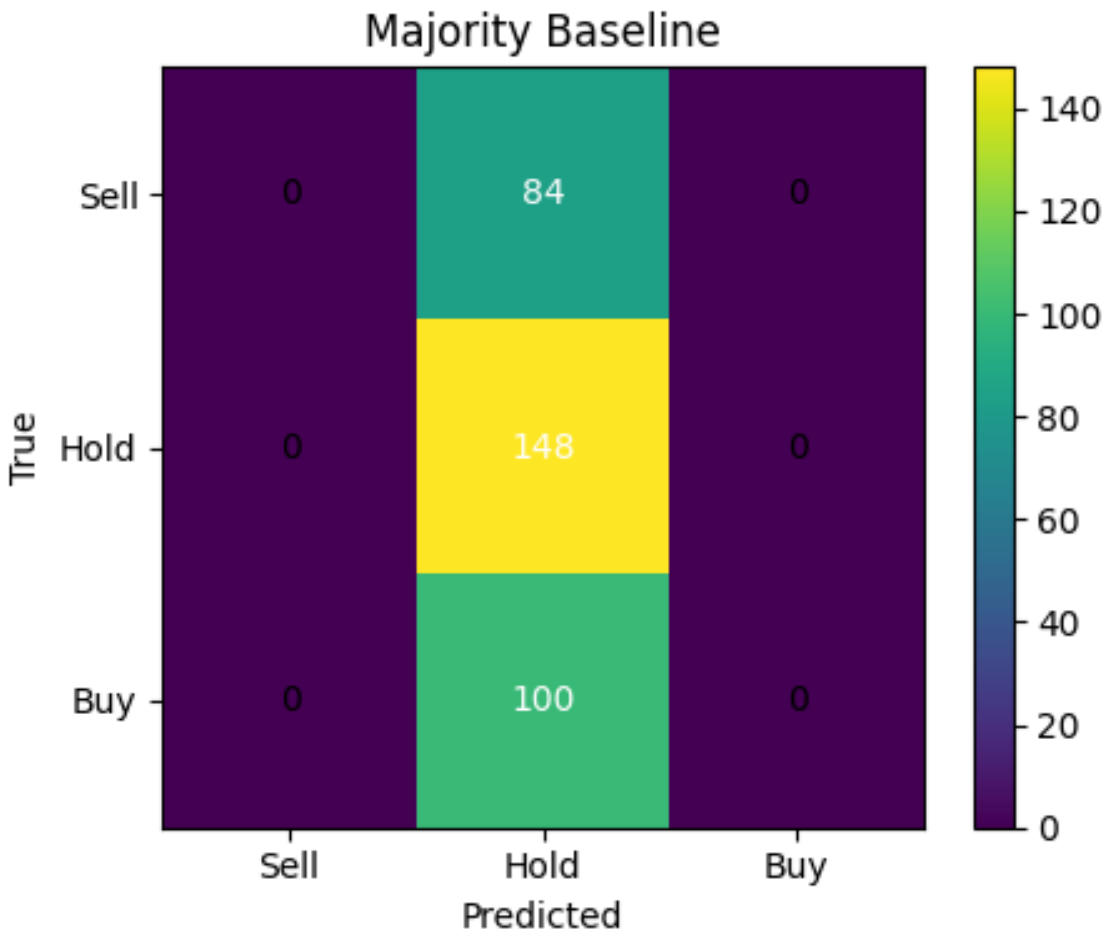


Figure 13: Confusion matrix of the majority-class baseline on the test set.

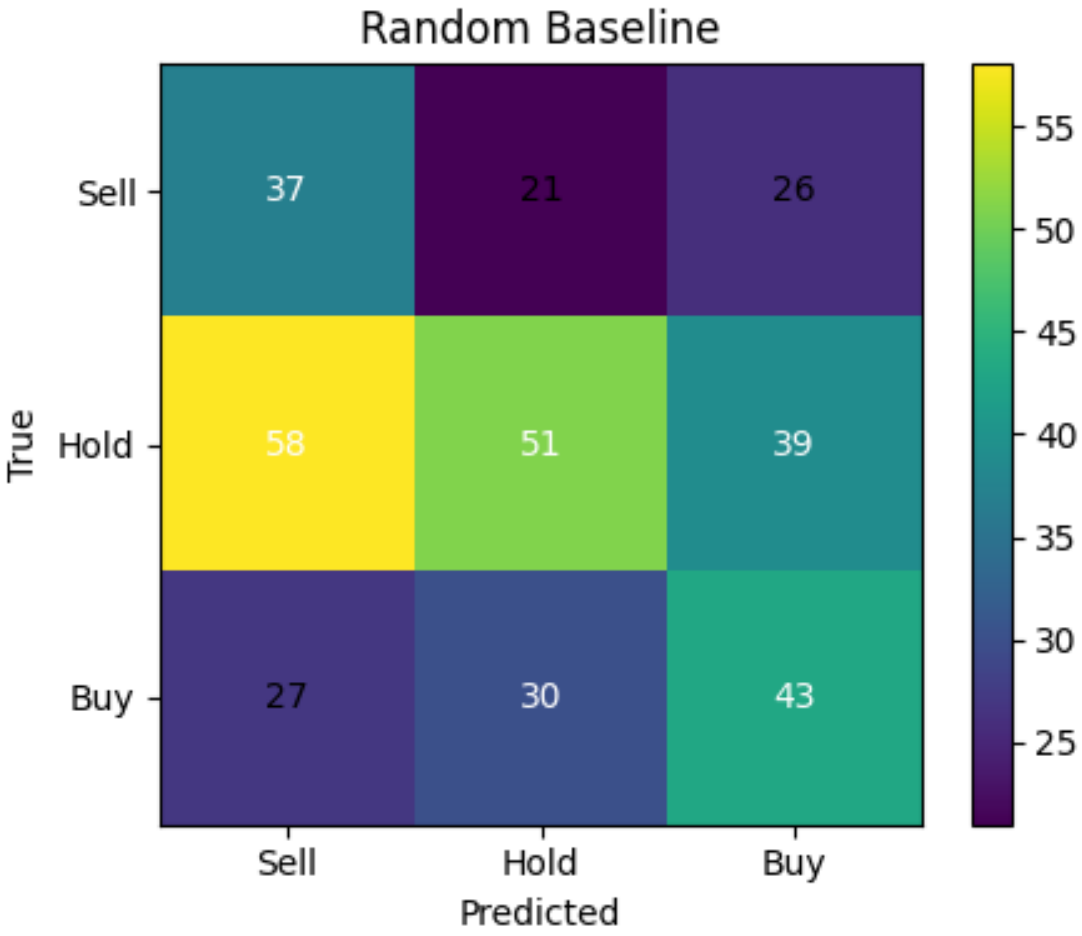


Figure 14: Confusion matrix of the random baseline on the test set.